INNOMATICS RESEARCH LABS

Deliverable 2 of 3

Low-Level Design Document

RAG-Based Customer Support Assistant with LangGraph and HITL

Project:	RAG-Based Customer Support Assistant
Prepared By:	Shravan Shidruk
Institute:	Innomatics Research Labs
Tech Stack:	Python, LangChain, LangGraph, ChromaDB, Ollama, Groq
Date:	April 2026

Table of Contents

1. Module-Level Design

The project is organized as a Python package under the app/ directory. Each subdirectory is a module with a single, clearly scoped responsibility. The modules communicate exclusively through function return values; there is no shared global state outside the LangGraph GraphState object.

1.1 Folder Structure

```
project/
app/
main.py # Entry point: pipeline setup + chat loop
main2.py # Step-by-step test runner
ingestion/
loader.py # PDF loading via PyPDFLoader
chunker.py # Text cleaning + recursive splitting
embedder.py # OllamaEmbeddings factory
retrieval/
retriever.py # ChromaDB store, retriever, dedup
llm/
llm.py # ChatGroq LLM factory
graph/
workflow.py # LangGraph StateGraph definition
routing/
router.py # Conditional routing logic
hitl/
escalation.py # Human-in-the-Loop handler
utils/
__init__.py # Reserved for shared utilities
data/
knowledge_base.pdf # Source knowledge base document
vectorstore/
chroma.sqlite3 # Persistent ChromaDB SQLite store
requirements.txt # Full pinned dependency list
```

1.2 Document Processing Module (ingestion/loader.py)

The load_pdf(file_path: str) function wraps LangChain's PyPDFLoader. It accepts a relative or absolute path to a PDF file and returns a list of Document objects. Each Document carries page_content (raw extracted text) and metadata (containing the source file path and page number). The function is deterministic and stateless.

```
from langchain_community.document_loaders import PyPDFLoader
```

```
def load_pdf(file_path: str):
loader = PyPDFLoader(file_path)
documents = loader.load()
return documents # List[Document]
```

1.3 Chunking Module (ingestion/chunker.py)

The module provides two functions. clean_text(text: str) normalizes whitespace by collapsing multiple spaces and newlines into single spaces, and removes pipe characters that appear as PDF table artifacts. chunk_documents(documents) applies clean_text to every document, then instantiates RecursiveCharacterTextSplitter with chunk_size=700, chunk_overlap=150, and a hierarchy of separators (paragraph, line, sentence, word). The overlap of 150 characters ensures that sentences crossing chunk boundaries are represented in both adjacent chunks, reducing retrieval misses.

```
text_splitter = RecursiveCharacterTextSplitter(
chunk_size=700,
chunk_overlap=150,
separators=['\n\n', '\n', '. ', ' ']
)
chunks = text_splitter.split_documents(documents)
```

1.4 Embedding Module (ingestion/embedder.py)

get_embedding_model() returns an OllamaEmbeddings instance pointing to the nomic-embed-text model at http://localhost:11434. This model produces 768-dimensional dense vectors optimized for semantic similarity tasks. The function acts as a factory and is called once during pipeline setup; the returned object is passed into both store_embeddings() and get_retriever() to ensure consistency between stored and query-time embeddings.

1.5 Vector Storage Module (retrieval/retriever.py)

store_embeddings(chunks, embedding_model) calls Chroma.from_documents(), which embeds each chunk and writes the vectors to the persist_directory='vectorstore' path. On subsequent runs the same directory is reused, meaning the ingestion step can be skipped if the knowledge base has not changed.

1.6 Retrieval Module (retrieval/retriever.py)

get_retriever(embedding_model) loads the existing ChromaDB collection and calls as_retriever() with search_type='mmr' and search_kwargs={'k': 3, 'fetch_k': 6}. MMR first retrieves fetch_k=6 candidates by cosine similarity, then re-ranks them to maximize diversity while maintaining relevance, returning the top k=3 chunks. retrieve_docs(query, retriever) adds a deduplication step using a set of seen content strings, preventing duplicate chunks from inflating the context window.

1.7 Query Processing Module (graph/workflow.py - process_node)

The process_node() function is the first node in the LangGraph workflow. It extracts query, retriever, and llm from the GraphState, invokes the retriever, assembles the retrieved chunks into a context string, constructs a structured prompt that restricts the LLM to context-only answers, and calls llm.invoke(prompt). It returns an updated state containing the LLM-generated answer.

1.8 Graph Execution Module (graph/workflow.py - build_graph)

build_graph() constructs and compiles the StateGraph. The four nodes (process, decision, respond, escalate) are added, the entry point is set to process, and a conditional edge is added from decision using a lambda that reads state['decision']. The compiled graph is returned as a runnable object.

1.9 HITL Module (hitl/escalation.py)

human_intervention(query) prints the escalated query to stdout and calls Python's built-in input() to block execution until a human operator types a response. The typed string is returned to the escalation_node in the graph, which stores it as state['answer'].

2. Data Structures

2.1 Document Representation

LangChain's Document dataclass is used throughout the pipeline.

```
class Document:
page_content: str # Raw or cleaned text content of the chunk/page
metadata: dict # {'source': 'data/knowledge_base.pdf', 'page': N}
```

2.2 Chunk Format

After chunking, each Document has a page_content of at most 700 characters (before the 150-char overlap addition). The metadata dict is preserved from the original page-level Document, so each chunk retains its source page reference.

2.3 Embedding Structure

nomic-embed-text produces 768-dimensional float32 vectors. ChromaDB stores each vector alongside its document text and metadata in the SQLite-backed collection. The collection UUID is 65543c6e-eeac-43c3-a0bd-2bd4a056ad43 (as seen in the vectorstore directory).

2.4 Query-Response Schema

```
Input -> query: str (user's natural language question)
Output -> answer: str (LLM-generated or human-provided response)
```

2.5 GraphState - State Object for LangGraph

The GraphState is defined as a TypedDict with total=False, meaning all keys are optional at initialization. This allows nodes to add keys incrementally as the workflow progresses.

```
from typing import TypedDict
class GraphState(TypedDict, total=False):
query: str # The original user question
answer: str # The generated or human-provided answer
retriever: object # The active ChromaDB MMR retriever instance
llm: object # The ChatGroq LLM instance
decision: str # Routing outcome: 'respond' or 'escalate'
```

3. Workflow Design - LangGraph

The LangGraph StateGraph defines a directed graph where each node is a Python function that receives the current state, performs its operation, and returns a partial state update. LangGraph merges the returned dict into the running state before passing it to the next node.

3.1 Nodes

Node Name	Function	Input Keys Used	Output Keys Written
process	`process_node()`	query, retriever, llm	query, answer, retriever, llm
decision	`decision_node()`	query, answer	decision
respond	`output_node()`	full state	full state (pass-through)
escalate	`escalation_node()`	query, retriever, llm	query, answer, retriever, llm

Table 4: LangGraph Node Definitions

3.2 Edges and State Transitions

The graph has the following edge structure:

- process -> decision (unconditional edge via graph.add_edge)
- decision -> respond (conditional, when state['decision'] == 'respond')
- decision -> escalate (conditional, when state['decision'] == 'escalate')
- respond -> END (LangGraph terminal node, implicit)
- escalate -> END (LangGraph terminal node, implicit)

The entry point is set to the process node via graph.set_entry_point('process'). Conditional edges are registered with graph.add_conditional_edges(), passing a lambda that extracts the decision key from the state.

4. Conditional Routing Logic

The route_decision() function in routing/router.py implements the routing policy. It operates on two inputs: the query string and the answer string, both lowercased before comparison.

4.1 Answer Generation Criteria (respond path)

A query is routed to the respond node when none of the escalation conditions are met. This means: the query does not contain escalation keywords, and the LLM produced an answer longer than 30 characters that does not contain the phrase 'not found'.

4.2 Escalation Criteria

- Keyword match on query: If 'refund' appears in the query, escalate immediately. Refund requests require human judgment on eligibility and policy exceptions.
- Keyword match on query: If 'payment issue' appears in the query, escalate immediately. Payment issues may involve sensitive financial data outside the knowledge base.
- Low confidence indicator: If the LLM answer contains 'not found' or 'i don\'t know', this signals the retrieval step did not surface relevant context.
- Short answer heuristic: If len(answer) < 30 characters, the LLM likely produced a non-answer, indicating missing context or a failed retrieval.

4.3 Routing Code Reference

```
def route_decision(state):
query = state['query'].lower()
answer = state['answer'].lower()

if 'refund' in query:
return 'escalate'
if 'payment issue' in query:
return 'escalate'
if 'not found' in answer or len(answer) < 30:
return 'escalate'

return 'respond'
```

5. HITL Design

5.1 When Escalation is Triggered

Escalation is triggered programmatically by the route_decision() function. The three triggering conditions are: a refund-related query, a payment-issue-related query, or a low-confidence LLM answer. When any of these conditions are true, decision_node() writes 'escalate' into state['decision'], causing LangGraph's conditional edge to route to the escalation_node().

5.2 What Happens After Escalation

The escalation_node() calls human_intervention(query), which prints a visible notification to the terminal and displays the user's original query. The function then blocks on input(), waiting for the human operator to type a free-form response and press Enter. There is no timeout in the current implementation.

5.3 How Human Response is Integrated

The string entered by the human operator is returned from human_intervention() and stored as state['answer'] in the updated state returned by escalation_node(). This human response is then displayed to the user through the same output path as a normal LLM answer, ensuring a seamless experience for the end user.

```
def human_intervention(query):
print('\n Escalation Triggered!')
print(f'User Query: {query}')
human_response = input('Enter human response: ')
return human_response
```

5.4 Current Limitations

- The HITL module is synchronous and CLI-based; it blocks the entire program until a human responds.
- There is no logging or audit trail of escalated queries and human responses.
- There is no timeout or fallback if the human does not respond.
- In a production system, escalation would write to a ticket queue (e.g., Jira, Zendesk) and resume asynchronously.

6. API and Interface Design

6.1 Input Format

The system accepts a plain string query from the CLI. The input is read via Python's input() function in the run_chatbot() loop in main.py. The query is then passed as state['query'] into graph.invoke().

```
graph.invoke({
'query': query, # str - user's natural language question
'retriever': retriever, # object - ChromaDB MMR retriever
'llm': llm # object - ChatGroq LLM instance
})
```

6.2 Output Format

The graph returns the final GraphState dict. The answer key contains the string to display. The main.py loop accesses result['answer'] and prints it with a separator line between turns.

6.3 Interaction Flow

- User types a query at the 'You:' prompt.
- The query is validated (empty queries implicitly pass to the graph).
- graph.invoke() is called synchronously.
- The result['answer'] string is printed under the 'Assistant:' label.
- A dashed separator is printed and the loop repeats.
- Typing 'exit' or 'quit' (case-insensitive) breaks the loop and exits.

7. Error Handling

Error Scenario	Where it Occurs	Current Behavior	Recommended Handling
PDF file not found	loader.py	PyPDFLoader raises FileNotFoundError and crashes.	Wrap in try/except; log and exit gracefully with a user message.
No relevant chunks found	retriever.py	Empty list returned; context is empty string in prompt.	Check len(docs)==0 before prompt construction; set answer='not found' to trigger escalation.

LLM API failure (Groq)	workflow.py process_node	Exception propagates and crashes the graph.	Wrap llm.invoke() in try/except; return a fallback 'service unavailable' answer and escalate.
Ollama not running	embedder.py	Connection refused error at embedding step.	Catch ConnectionRefusedError; prompt user to start Ollama before retrying.
ChromaDB store missing	retriever.py get_retriever	Chroma loads an empty collection; retriever returns no results.	Check for existence of vectorstore/ directory; run ingestion if absent.

Table 5: Error Scenarios and Handling

End of Deliverable 2 - Low-Level Design